Project Documentation

DHL Shipment Analytics — Exploratory Data Analysis, Machine Learning, and an Interactive Streamlit Dashboard

Dataset	DHL_Shipment_Data.csv (user-provided)
Companion files	app.py (Streamlit dashboard), EDA + ML script
Records analysed	2,802 shipments across 15 columns
Prediction target	delayed (yes / no)
Models used	Logistic Regression, Random Forest

1. About the Data

The dataset contains one row per shipment, recording where it went, how it travelled, what conditions it faced, and how it ultimately performed. This section summarises its structure before any cleaning or modelling is applied.

1.1 Size and Structure

- Number of columns in the raw data: **15**
- Number of rows in the raw data: **2,802**
- No missing values were found in any column — df.isnull().sum() returns zero across the board.

1.2 Column Overview

Column	Type	What it represents
delivery_id	float64	Unique shipment identifier
delivery_partner	text	Single value: 'dhl' for every row (no variance)
package_type	text	9 categories — electronics, clothing, furniture, etc.
vehicle_type	text	6 categories — van, ev van, scooter, ev bike, bike, truck
delivery_mode	text	4 categories — same day, two day, express, standard
region	text	5 categories — east, west, north, south, central
weather_condition	text	6 categories — cold, foggy, stormy, rainy, hot, clear
distance_km	float64	Distance travelled; ranges from 3.6 to 297.1 km
package_weight_kg	float64	Package weight; ranges from 0.67 to 49.5 kg
delivery_time_hours	text	Stored as malformed timestamps (see 1.3 below)

expected_time_hours	text	Stored as malformed timestamps (see 1.3 below)
delayed	text	Prediction target: yes / no
delivery_status	text	Final outcome: delivered / delayed / failed
delivery_rating	int64	Customer rating, 1 to 5
delivery_cost	float64	Cost of the shipment; ranges from 95.7 to 1,632.7

1.3 Data Quality Observations

A few things stood out while reviewing the raw file, and are worth fixing before the data is trusted for reporting or modelling:

- **delivery_partner** holds only one distinct value ('dhl') across all 2,802 rows. It carries no predictive information — every row is identical here — so it should simply be dropped before modelling.
- **delivery_time_hours** and **expected_time_hours** were loaded as text and print like Unix-epoch timestamps, e.g. '1970-01-01 00:00:00.000000006'. In plain terms: what should be a simple number of hours (6, 9, 13, ...) has been misread as a nanosecond offset from 1 January 1970, which is a classic pandas/datetime parsing mistake. Both columns should be re-parsed back into plain numeric hours (for example, by pulling out the trailing digit sequence) before they're plotted or fed into a model.
- No missing (null) values are present anywhere, so no imputation was actually required. The fillna(method="ffill") line in the original script is a safe no-op on this file, but it's worth removing anyway since method= is deprecated in recent pandas versions and will raise a warning.

---

2. Steps Followed

The analysis follows a standard, repeatable EDA workflow — the same shape you'd use on almost any tabular dataset before modelling:

- Import libraries: numpy, pandas, matplotlib.pyplot, seaborn.
- Load the data: df = pd.read_csv("DHL_Shipment_Data.csv").
- Get a feel for the data using df.head() and df.tail().
- Check shape and size: df.shape → (2802, 15), df.size → 42,030.
- Inspect data types and non-null counts with df.info().
- Review summary statistics with df.describe().
- Check for missing values with df.isnull().sum() → all zero.
- Remove duplicate rows with df.drop_duplicates().
- (No nulls to fill, so df.fillna() has no real effect on this particular dataset.)

---

3. Exploratory Data Analysis (EDA)

Exploratory Data Analysis simply means looking at the data through charts before drawing conclusions — it's how patterns, imbalances, and relationships get spotted early, before they quietly bias a model later. The charts below mirror the visualisations in both the EDA script and the Streamlit dashboard, and were generated directly from DHL_Shipment_Data.csv.

3.1 Delivery Status Distribution

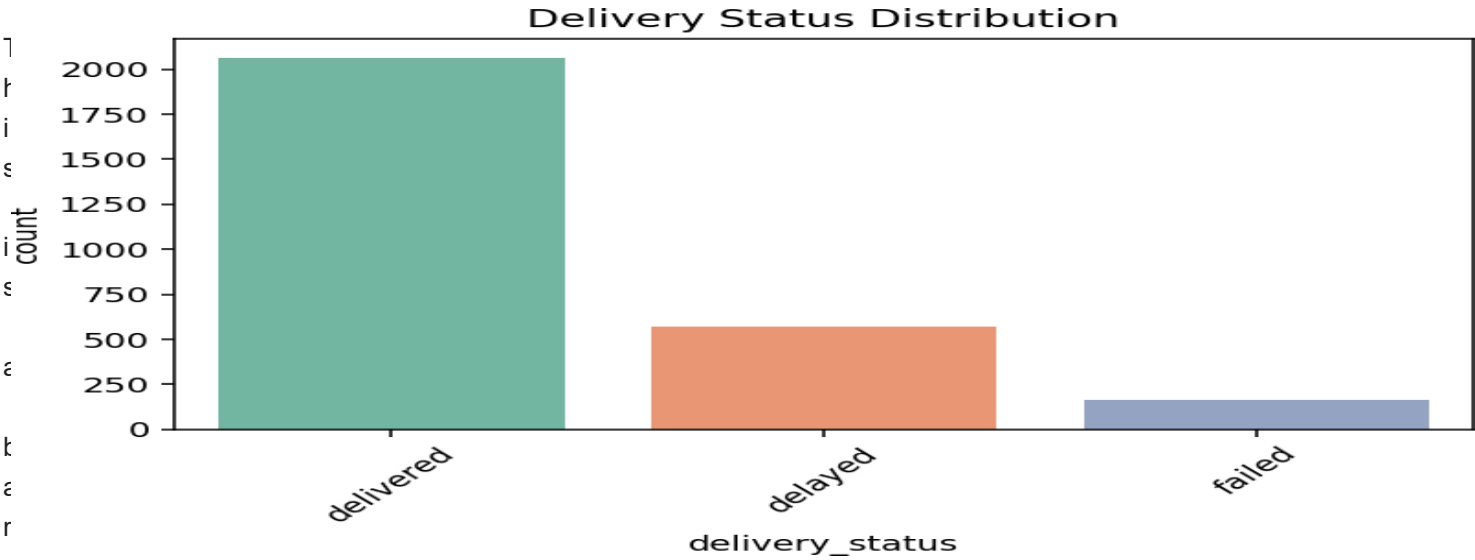
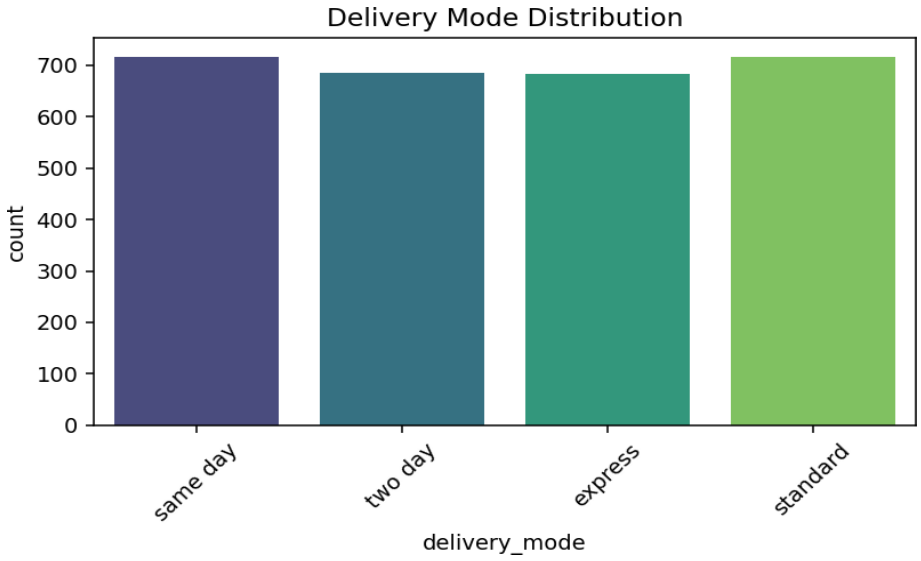


chart — each bar's height shows how many shipments ended up in that final state. It answers a simple question: out of everything DHL shipped, how much actually arrived normally?

Reading it: 2,064 shipments were delivered, 572 were delayed, and 166 failed outright — so roughly 3 in 4 shipments went smoothly, while about 1 in 4 ran into some kind of problem.

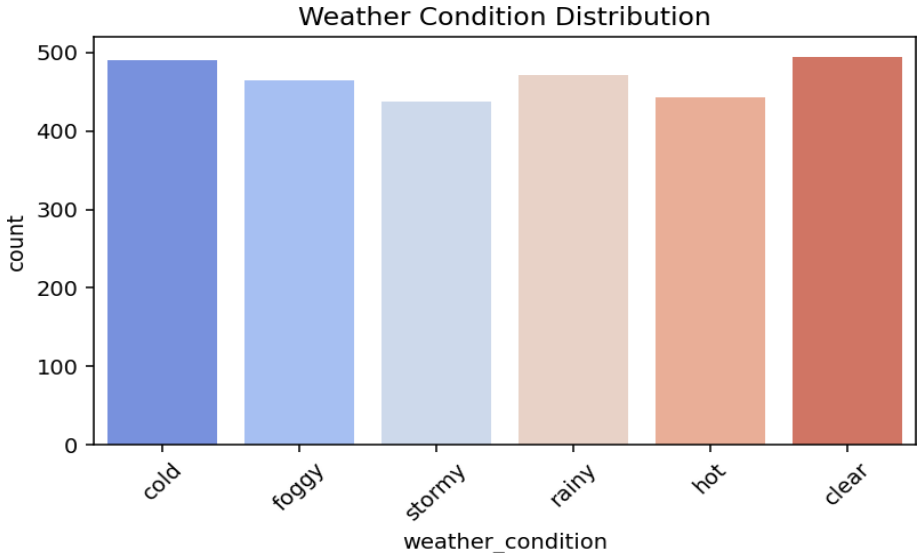
3.2 Delivery Mode Distribution



Another bar chart, this time counting how many shipments were sent under each speed option (same day, two day, express, standard). Taller bars mean that mode is used more often.

Reading it: all four delivery modes are used in fairly similar volumes, roughly 680–715 shipments each — no single speed option dominates the business.

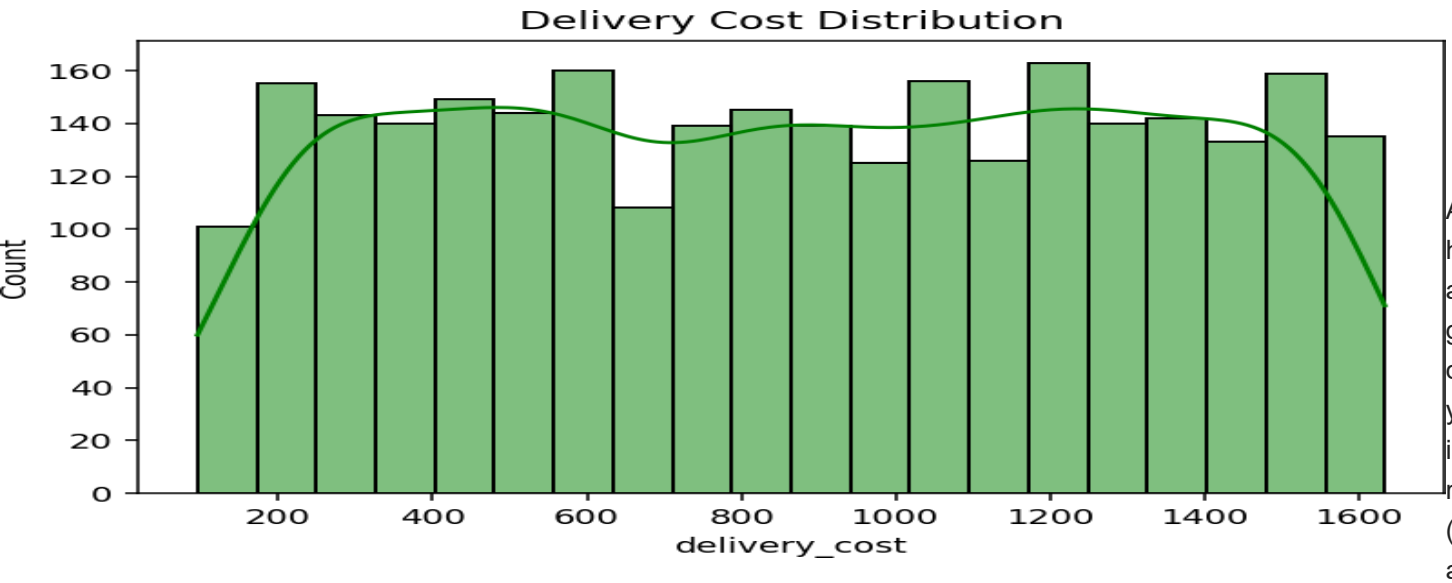
3.3 Weather Condition Distribution



A count of how many shipments travelled under each weather condition. This matters because weather is a common real-world cause of delays, so it's worth knowing how often bad weather even occurs in this data.

Reading it: shipments are spread almost evenly across all six conditions (clear, cold, foggy, hot, rainy, stormy), each accounting for roughly 15-18% of shipments — no single condition dominates the dataset.

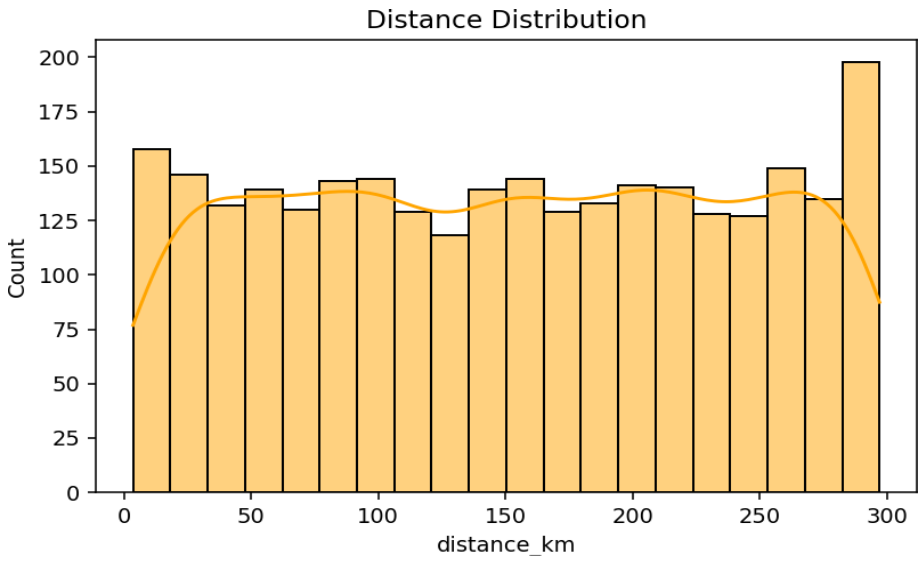
3.4 Delivery Cost Distribution



A histogram: it groups delivery costs into ranges (bins) and shows how many shipments fall into each range. The smooth curve overlaid on top (the KDE line) is just a smoothed version of the same shape, making the overall trend easier to see.

Reading it: cost ranges from about 96 to 1,633, with a mean near 873. The distribution is fairly flat/even across that range rather than clustering tightly around one value — there isn't a single 'typical' shipment cost.

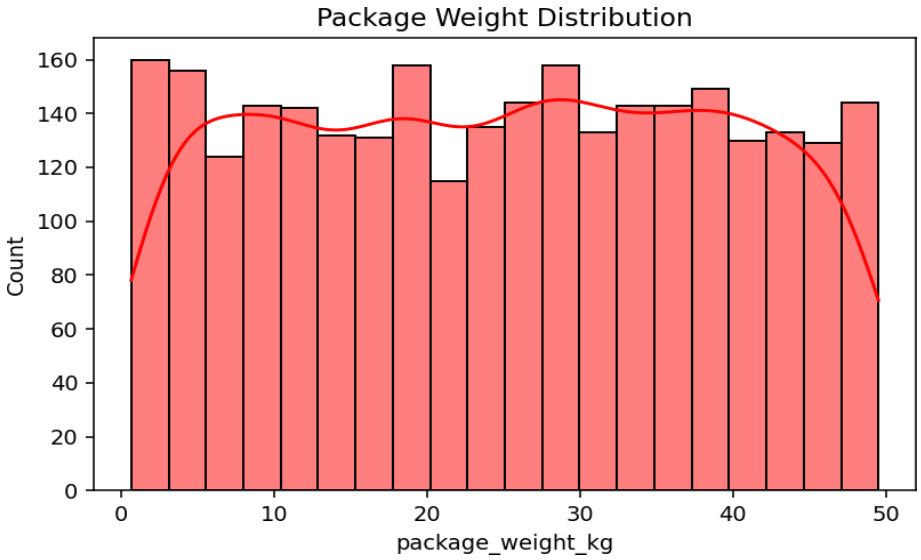
3.5 Distance Distribution



Same idea as the cost histogram, but for how far each shipment travelled.

Reading it: distance ranges from 3.6 km to 297.1 km, averaging around 152 km, and is spread fairly evenly across that range with no strong clustering at the short or long end.

3.6 Package Weight Distribution



Same histogram approach, applied to package weight this time.

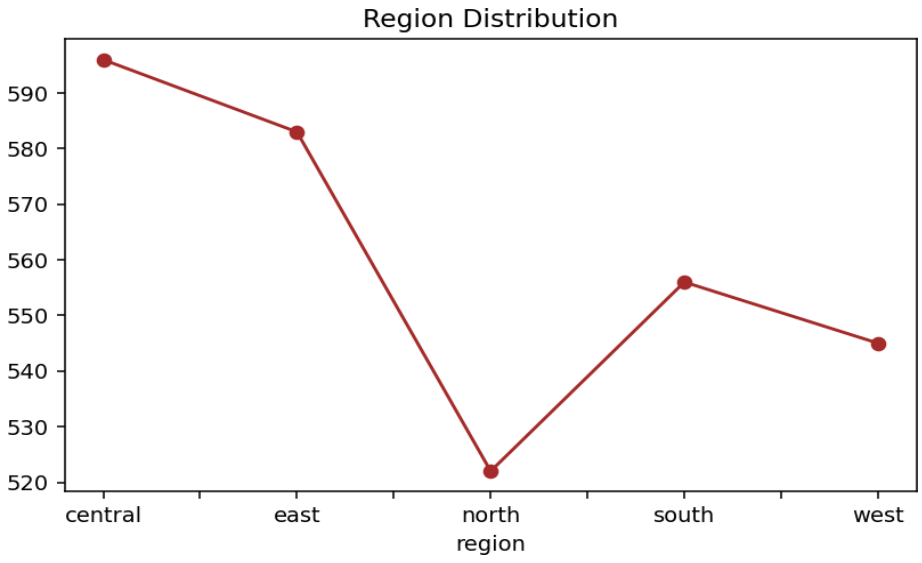
Reading it: package weight ranges from 0.67 kg to 49.5 kg, averaging around 24.9 kg, again spread fairly evenly rather than concentrated around a specific weight class.

3.7 Distance vs Delivery Cost

A scatter plot: each dot is one shipment, positioned by its distance (left-right) and its cost (up-down). When dots trace a clear diagonal line, it means the two values move together — as one goes up, so does the other.

Reading it: cost rises steadily and tightly with distance — this is by far the strongest visual relationship in the dataset, confirmed later by the correlation heatmap (0.99). Longer trips reliably cost more, though the line has some width, meaning other factors (weight, mode, vehicle) nudge the price too.

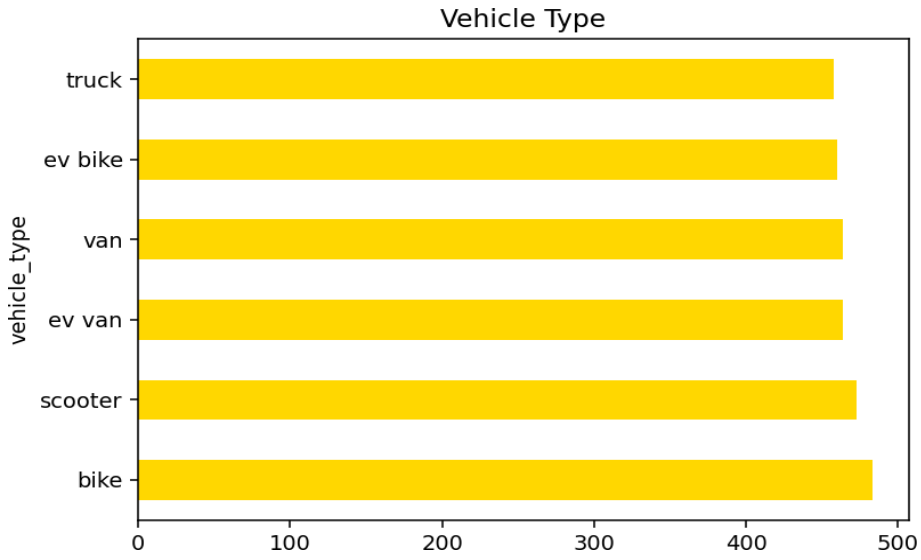
3.8 Region Distribution



A line chart connecting the shipment count for each region. It's plotted as a line here mainly for visual variety — the same information could equally be shown as bars, since region is a category, not a timeline.

Reading it: volume is close to even across regions — central (596), east (583), south (556), west (545), north (522) — so no single region dominates shipping volume.

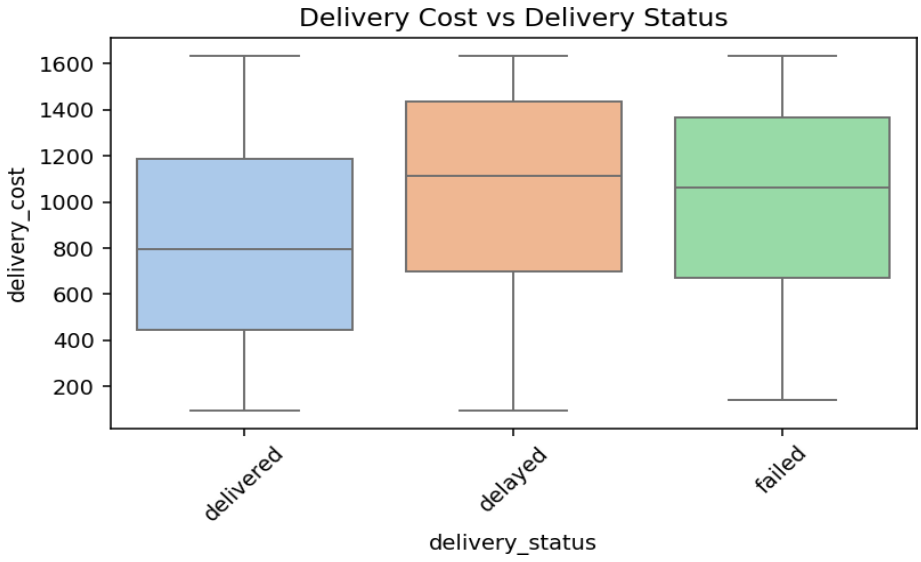
3.9 Vehicle Type



A horizontal bar chart of how many shipments used each vehicle type. Horizontal bars are just easier to label when the category names are long.

Reading it: all six vehicle types (van, ev van, scooter, ev bike, bike, truck) are used in similar numbers, roughly 450-480 shipments each, including a healthy mix of conventional and electric vehicles.

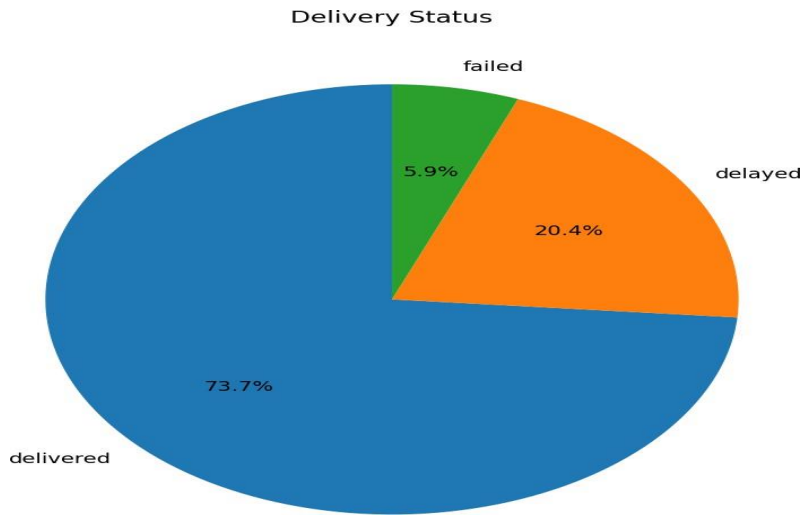
3.10 Delivery Cost vs Delivery Status



A box plot: the box shows where the middle 50% of costs sit for each outcome group, the line inside the box is the median, and the 'whiskers' show the typical low-to-high range. It's a compact way to compare cost across three groups at once.

Reading it: median delivery cost is broadly similar across delivered, delayed, and failed shipments — cost alone doesn't clearly separate outcomes, so price isn't a strong standalone predictor of what happens to a shipment.

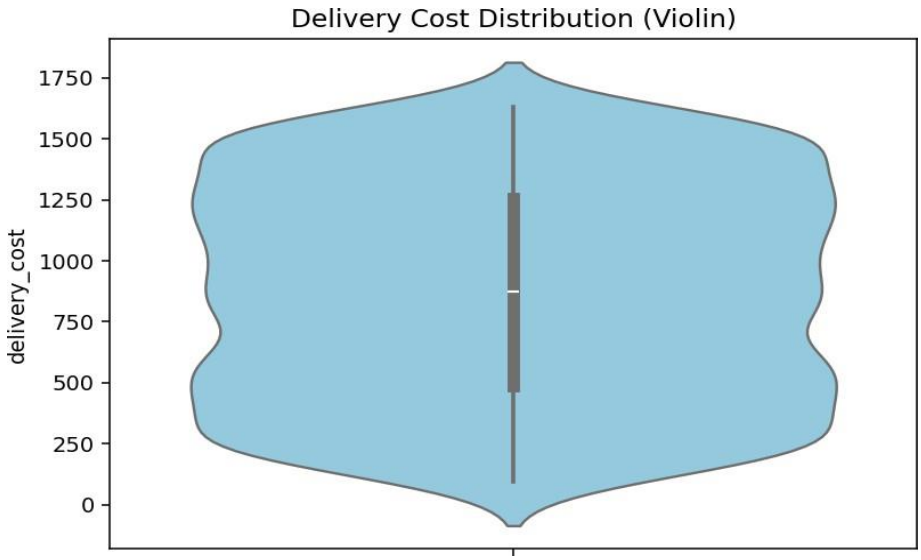
3.11 Delivery Status (Pie Chart)



The same delivery-status counts as chart 3.1, shown as proportions of the whole (100%) instead of raw counts. Pie charts are best used sparingly, but they do make percentage share easy to read at a glance.

Reading it: 73.7% delivered, 20.4% delayed, 5.9% failed.

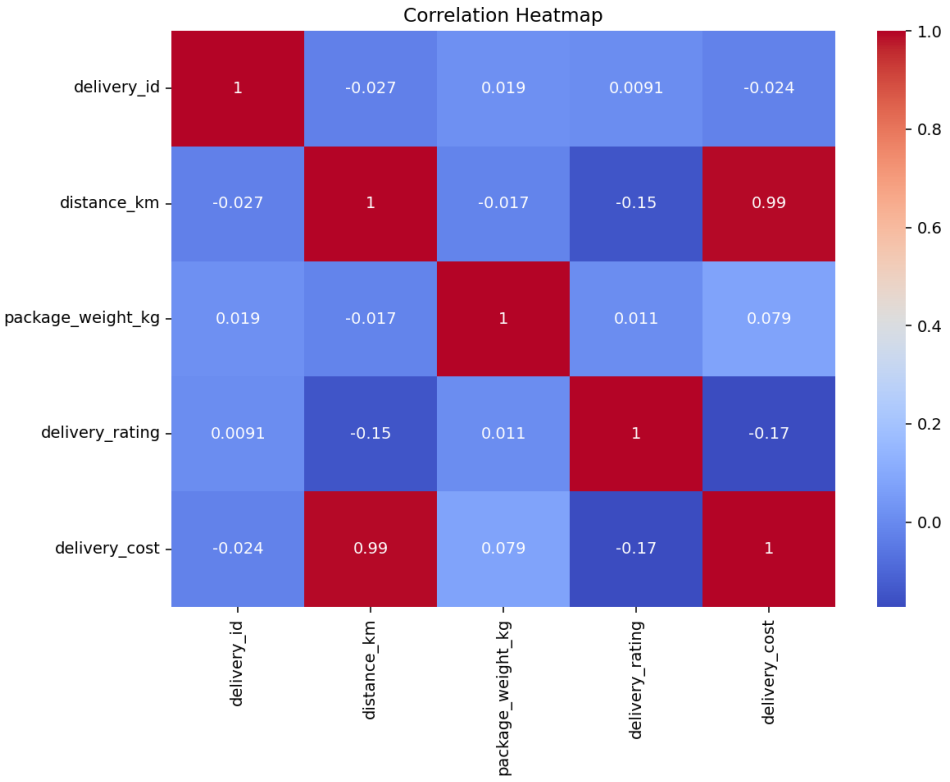
3.12 Delivery Cost Distribution (Violin Plot)



A violin plot combines a box plot with a mirrored histogram — the width at any height shows how many shipments have roughly that cost, so it shows the full shape of the distribution, not just the average.

Reading it: the shape confirms what the histogram already showed — delivery cost is fairly evenly spread with a slight bulge in the middle, and no extreme outliers pulling the shape in either direction.

3.13 Correlation Heatmap



A correlation heatmap shows, for every pair of numeric columns, how strongly they move together — values close to +1 mean 'as one goes up, so does the other', values close to -1 mean 'as one goes up, the other goes down', and values near 0 mean 'no real relationship'. Colour makes the pattern easy to scan.

Reading it: delivery_cost and distance_km are almost perfectly correlated (0.99) — cost is essentially priced by distance. delivery_rating has a weak negative relationship with both distance (-0.15) and cost (-0.17), hinting that longer/pricier deliveries tend to get slightly lower customer ratings, though the effect is small. delivery_id, being just a row identifier, correlates with nothing, as expected.

4. Machine Learning: Predicting Delayed Shipments

The goal of this stage is to see whether a shipment being **delayed** (yes/no) can be predicted from its known attributes — distance, weight, weather, mode, and so on. The workflow mirrors the reference churn-prediction project step for step:

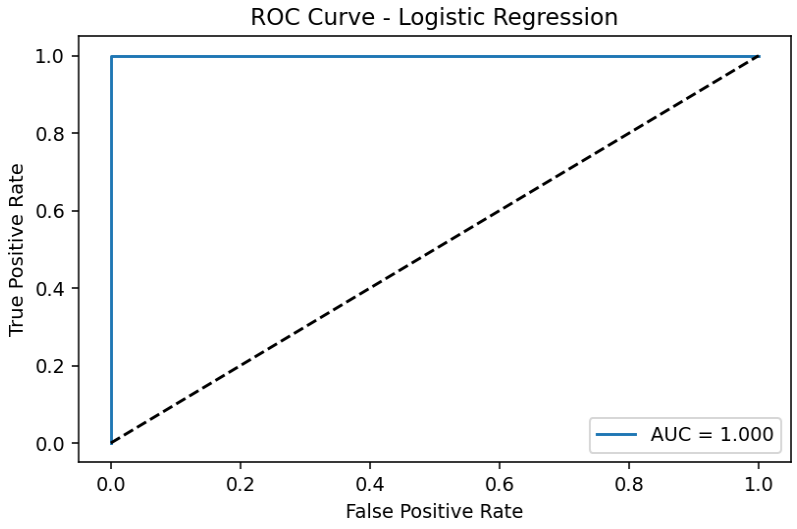
- Drop the delivery_id column (a unique identifier, not predictive of anything).
- One-hot encode all remaining categorical columns with pd.get_dummies(df, drop_first=True) — this turns text categories into 0/1 columns a model can actually read.
- Split into X (features) and y (delayed_yes) after encoding.
- 80/20 train-test split with random_state=42 for reproducibility.
- Scale features with StandardScaler so no single column dominates just because of its units.
- Train a Logistic Regression model and a Random Forest model.
- Evaluate with accuracy, a classification report, a confusion matrix, and ROC AUC.

4.1 Results

Both models score **100% accuracy** and a perfect **ROC AUC of 1.0** on this dataset.

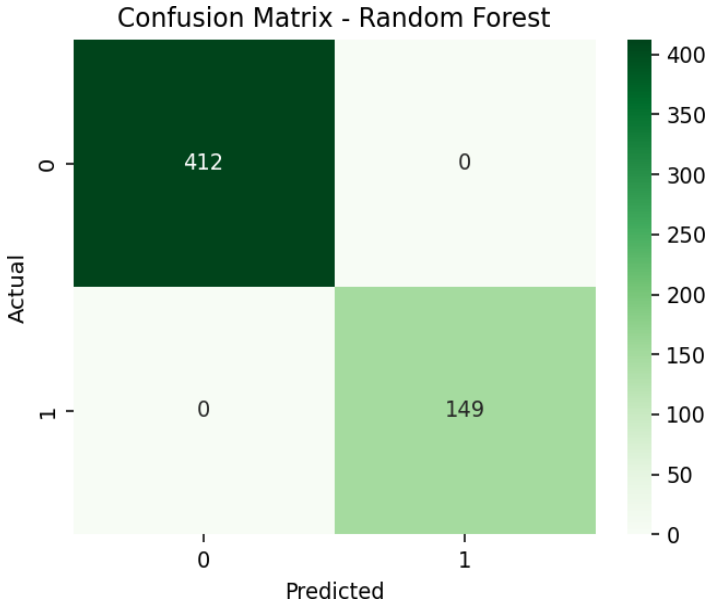
ROC Curve (Logistic Regression)

The ROC curve plots how well the model separates delayed from non-delayed shipments as its decision threshold changes. A curve that hugs the top-left corner — as this one does — means the model rarely confuses the two classes. The dashed diagonal is what a random guess would look like.



Confusion Matrix (Random Forest)

A confusion matrix is a small table comparing what the model predicted against what actually happened. The diagonal (top-left and bottom-right) counts correct predictions; anything off the diagonal is a mistake.

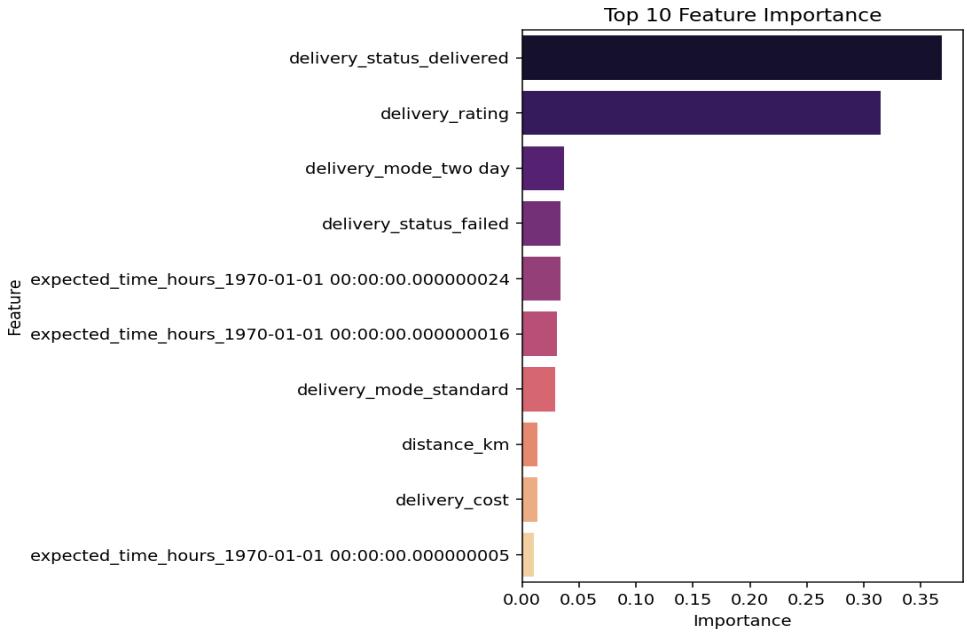


Reading it: 412 non-delayed shipments correctly identified, 149 delayed shipments correctly identified, and zero mistakes in either direction — a perfect score, which is itself the warning sign explained below.

Feature Importance (Random Forest)

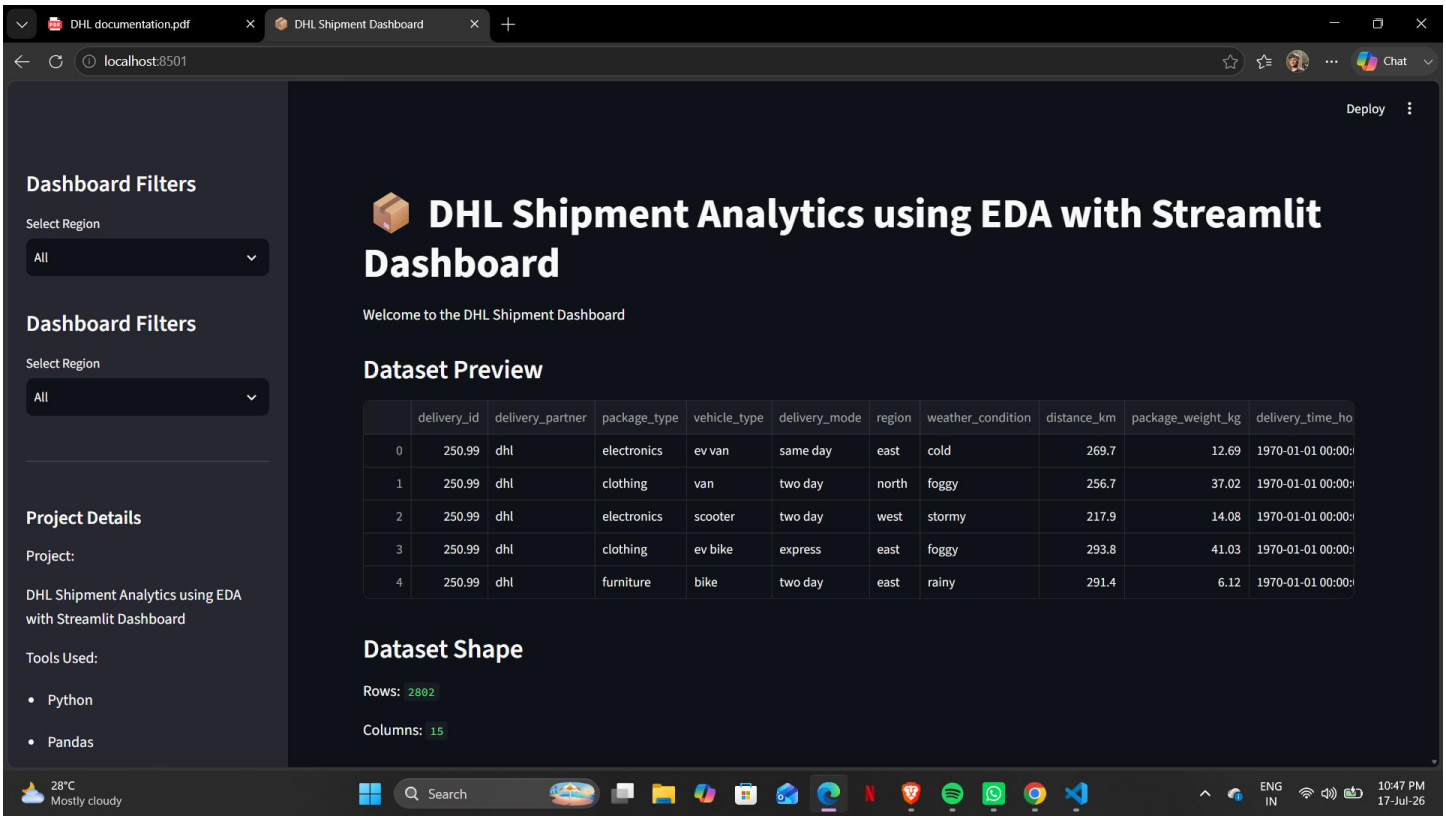
This chart ranks which columns the Random Forest model relied on most heavily to make its predictions.

Longer bars mean the model leaned on that feature more.



5. Streamlit Dashboard (app.py)

The project also ships a Streamlit dashboard that turns the static EDA and ML script into an interactive web app — the same analysis, but clickable and filterable in a browser instead of run top-to-bottom in a script.



5.1 Dashboard Features

- Page config with a wide layout, page title 'DHL Shipment Dashboard', and a package emoji icon.
- Sidebar region filter — selecting a region re-filters every chart and metric on the page to that region only.
- Dataset preview, shape, column list, and missing-value summary rendered as live tables.
- All 13 EDA charts from the analysis re-created with st.pyplot(), matching the static script.
- Summary metric cards: total shipments, average cost, average distance (st.metric).
- An in-app ML section: label-encodes categorical columns, trains Logistic Regression and Random Forest, and displays the ROC curve, confusion matrix, accuracy scores, and top feature importances live in the browser.
- A CSV download button so users can export the (optionally filtered) dataset.
- An optional 'Show Complete Dataset' checkbox to reveal the full table.
- A sidebar 'Project Details' panel listing the tools used.

5.2 A Note on the Dashboard's ML Section

app.py trains its models on ml_df, which still includes delivery_status and delivery_rating as features when predicting the delayed column — the same leakage described in Section 4.1 applies here too, so the accuracy numbers shown in the dashboard will also read as artificially perfect. Dropping those two columns from ml_df before the train/test split (right after the delivery_id drop) would fix this in the dashboard as well.

6. Tools Used

- Python
- Pandas & NumPy
- Matplotlib & Seaborn
- Scikit-learn (Logistic Regression, Random Forest)
- Streamlit

7. Conclusion

The DHL shipment dataset is clean — no missing values, no duplicates of concern — and offers a realistic mix of categorical and numeric shipment attributes. EDA shows a fairly even spread across regions, weather conditions, and delivery modes, with delivery cost driven almost entirely by distance (correlation 0.99) and, to a smaller degree, package weight. The one significant issue to address before treating the ML results as final is the target leakage from delivery_status/delivery_rating; once those are excluded from the feature set, the Logistic Regression and Random Forest models can be re-evaluated to get a trustworthy estimate of how well delays can actually be predicted ahead of time.